



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: Lógica y diseño digital	Unidad #: 4	Tema: Circuitos combinacionales y secuenciales.
	Semana 12	Tutor:

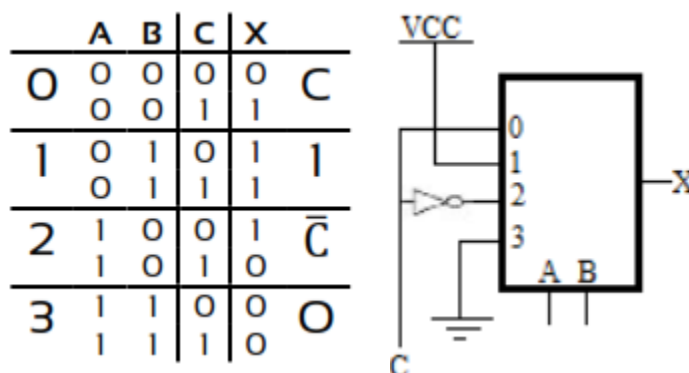
Introducción:

Bienvenidos al material de lectura #001 de la Unidad #4 de la materia Lógica y diseño digital. En este material realizaremos un repaso de los circuitos combinacionales siguientes: Multiplexor, Decodificador y Comparador; adicional estudiaremos los Codificadores y Demultiplexores.

Este material de lectura está diseñado para proporcionarte una base sólida en el campo de la ingeniería en desarrollo de software, preparándote para enfrentar los desafíos y oportunidades que encontrarás en tu carrera profesional. A medida que avances en este material, te alentamos a participar activamente de las actividades, para favorecer tu proceso de aprendizaje.

[illegible]

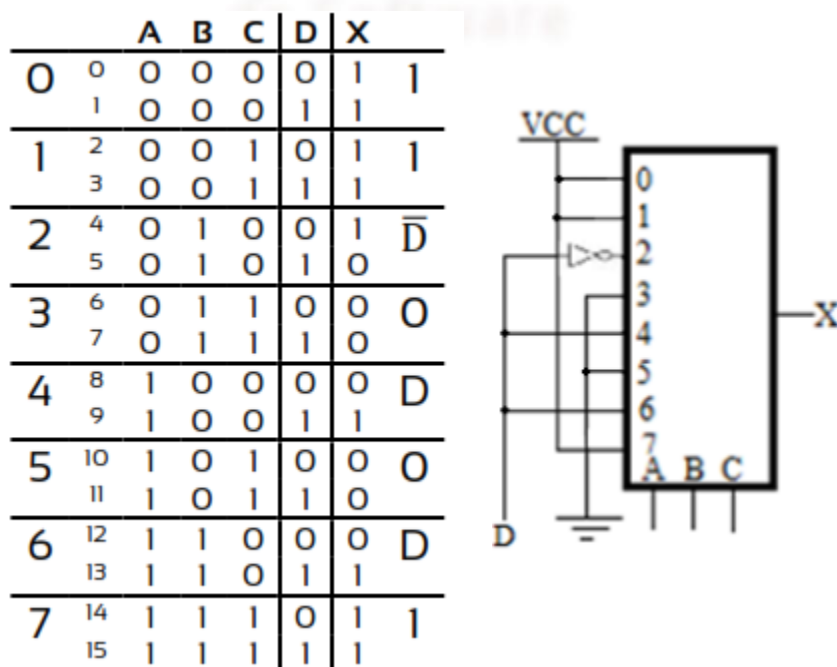
2. Cantidad de variables de selección del multiplexor menor que la cantidad de variables de la función lógica. Implementar X en un multiplexor de 4 a 1.



Para este caso en el cual la cantidad de variables de selección del multiplexor es menor que las variables de la función lógica, lo que se hace es, dividir la tabla de verdad en las variables que se necesiten (elegir las primeras variables), así formando la tabla de verdad para las entradas solicitadas, el funcionamiento de las variables adicionales se procede a diseñar con puertas.

Para el caso mostrado se eligieron las variables A y B para formar la tabla de verdad del multiplexor utilizado (ya que sus bits son iguales), para la variable C, se utilizó una compuerta NOT en la entrada de 2 para obtener su salida de la tabla de verdad, ya que, para el caso de la tabla en 1, cuando $C = 0$, $X = 1$ y cuando $C = 1$, $X = 0$, para las demás entradas no varía el valor de C con X.

Ejercicio. $X = \sum m(0, 1, 2, 3, 4, 9, 13, 14, 15)$, con multiplexores de 3 variables de entradas.



Codificadores.

Un codificador es un circuito digital que efectúa la operación inversa de la que efectúa un decodificador. El codificador tiene 2^n (o menos) líneas de entrada y n líneas de salida. Estas últimas generan el código binario correspondiente al valor de entrada.

Un ejemplo de codificador es el codificador de octal a binario cuya tabla de verdad se presenta en la Tabla 4-1. Tiene ocho entradas (una para cada uno de los dígitos octales) y tres salidas que generan el número binario correspondiente. Se supone que sólo una entrada es igual a 1 en cualquier momento dado.

El codificador se puede implementar con compuertas OR cuyas salidas se determinan directamente de la tabla de verdad. La salida Z es 1 cuando el dígito octal de entrada es 1, 3, 5 o 7. La salida Y es 1 para los dígitos octales 2, 3, 6 o 7, y la salida X es 1 para los dígitos 4, 5, 6 o 7. Estas condiciones se expresan con las funciones booleanas de salida siguientes:

$$Z = D_1 + D_3 + D_5 + D_7$$

$$Y = D_2 + D_3 + D_6 + D_7$$

$$X = D_4 + D_5 + D_6 + D_7$$

El codificador se implementa con tres compuertas OR.

Entradas								Salidas		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Tabla 4-1 Tabla de verdad del codificador de octal a binario.

El codificador definido en la tabla 4-1 tiene la limitación de que sólo una entrada puede estar activa en un momento dado. Si dos entradas están activas simultáneamente, la salida producirá una combinación no definida. Por ejemplo, si D_3 y D_6 son 1 simultáneamente, la salida del decodificador será 111 porque las tres salidas son 1. Esto no representa ni el 3 binario ni el 6 binario. Para resolver esta ambigüedad, los circuitos codificadores deben establecer una prioridad de entrada que garantice que sólo se codificará una de las entradas. Si establecemos que las entradas con subíndice más alto tienen prioridad, y si tanto D_3 como D_6 son 1 al mismo tiempo, la salida será 110 porque D_6 tiene prioridad sobre D_3 .

Otra ambigüedad en el codificador de octal a binario es que se genera una salida de tres ceros cuando todas las entradas son 0; esta salida es la misma que se produce cuando D₀ es igual a 1. La discrepancia se resuelve añadiendo una salida más que indique que por lo menos una entrada es 1.

Entradas				Salidas		
D ₀	D ₁	D ₂	D ₃	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

Tabla 4-2 Tabla de verdad de un codificador con prioridad.

Codificador con prioridad.

Un codificador con prioridad es un circuito codificador que incluye la función de prioridad. Su funcionamiento es tal que, si dos o más entradas son 1 al mismo tiempo, la salida prioritaria tendrá precedencia. En la tabla 4-2 se presenta la tabla de verdad de un codificador de cuatro entradas con prioridad. Además de las dos salidas, X y Y, el circuito tiene una tercera salida designada V; ésta es un indicador de bit válido que adquiere el valor 1 cuando una o más entradas son 1.

Si todas las entradas son 0, la entrada no será válida y V será 0. En tal caso, las otras dos salidas no se inspeccionarán y se especifican como condiciones de indiferencia. Advierta que, aunque las X en las columnas de salida representan condiciones de indiferencia, las X de las columnas de entrada sirven para representar una tabla de verdad en forma condensada. En vez de presentar los 16 minterminos de cuatro variables, la tabla de verdad usa una X para representar tanto 1 como 0. Por ejemplo, X100 representa los dos minterminos 0100 y 1100.

Según la tabla 4-2, cuanto más alto sea el subíndice de una entrada, mayor prioridad tendrá esa entrada. La entrada D₃ es la de mayor prioridad, así que, si es 1, la salida XY será 11 (3 binario) sin importar qué valor tengan las demás entradas. D₂ está en el siguiente nivel de prioridad. La salida es 10 si D₂=1, siempre que D₃=0 e independientemente del valor que tengan las otras dos entradas de menor prioridad. La salida para D₁ sólo se genera si las entradas con mayor prioridad son 0, y así hasta el nivel de prioridad más bajo.

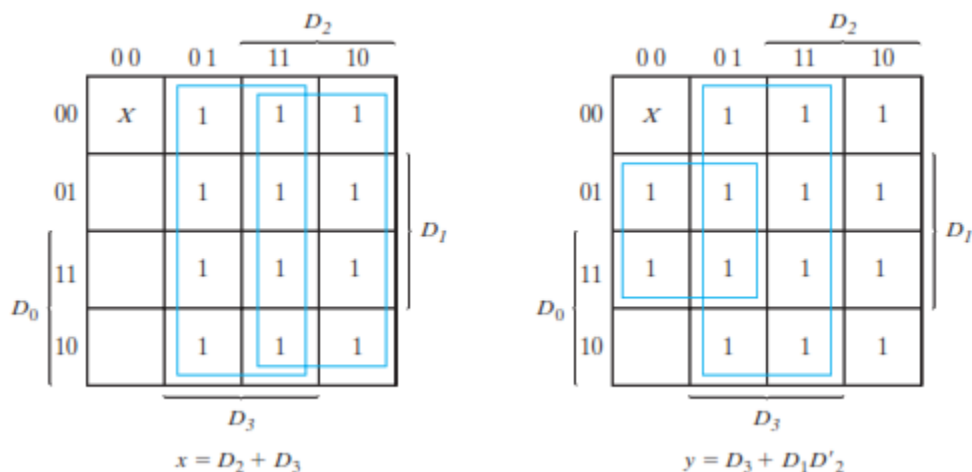


Figura 4-3 Mapas para el codificador con prioridad.

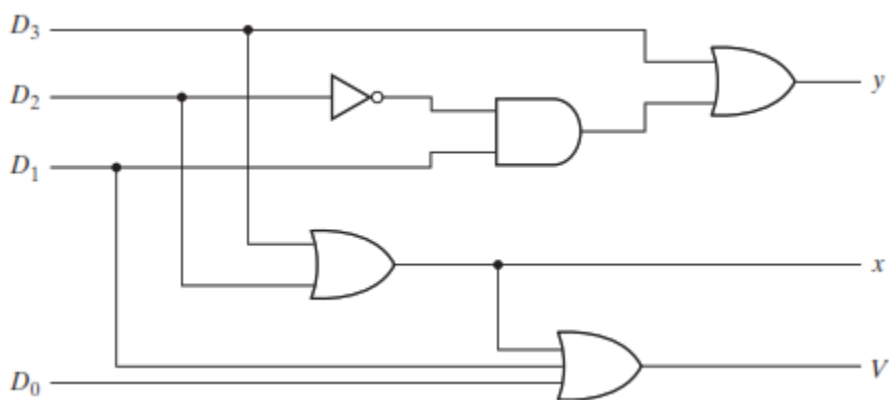


Figura 4-4 Codificador de cuatro entradas con prioridad.

Los mapas para simplificar las salidas X y Y aparecen en la figura 4-3. Los minterminos para las dos funciones se deducen a partir de la tabla 4-2. Aunque la tabla sólo tiene cinco filas, al sustituir cada X de una fila, primero por 0 y después por 1, obtendremos las 16 posibles combinaciones de entrada. Por ejemplo, la cuarta fila de la tabla, que tiene XX10, representa los cuatro minterminos 0010, 0110, 1010 y 1110. Las expresiones booleanas simplificadas para el codificador con prioridad se obtienen de los mapas. La condición para la salida V es una función OR de todas las variables de entrada. El codificador con prioridad se implementa en la figura 4-4 de acuerdo con las funciones booleanas siguientes:

$$X = D_2 + D_3$$

$$Y = D_3 + D_1 D'_2$$

$$V = D_0 + D_1 + D_2 + D_3$$

Decodificadores y Demultiplexores.

Decodificadores. En los sistemas digitales, las cantidades discretas de información se representan con códigos binarios. Un código binario de n bits puede representar hasta 2^n elementos distintos de información codificada. Un decodificador es un circuito combinacional que convierte información binaria de n líneas de entrada a un máximo de 2^n líneas de salida distintas. Si la información codificada en n bits tiene combinaciones que no se usan, el decodificador podría tener menos de 2^n salidas.

Algunos decodificadores se construyen con compuertas NAND. Puesto que una compuerta NAND produce la operación AND con la salida invertida, resulta más económico generar los minterminos del decodificador en su forma complementada. Además, los decodificadores incluyen una o más entradas habilitadoras (enable) que controlan el funcionamiento del circuito. En la figura 4-5 se aprecia un decodificador de 2 a 4 líneas con entrada de habilitación, construido con compuertas NAND. El circuito opera con salidas complementadas y una entrada de habilitación complementada. El decodificador se habilita cuando $E=0$. Como indica la tabla de verdad, sólo una salida puede ser 0 en cualquier momento dado; todas las demás salidas son 1. La salida cuyo valor es 0 representa el mintermino seleccionado por las entradas A y B. El circuito queda inhabilitado cuando $E=1$, sean cuales sean los valores de las otras dos entradas. Cuando el circuito está inhabilitado, ninguna de las salidas es 0 y ninguno de los minterminos está seleccionado. En general, un decodificador podría operar con salidas complementadas o no complementadas. La entrada de habilitación podría activarse con una señal 0 o con una señal 1. Algunos decodificadores tienen dos o más entradas de habilitación que deben satisfacer una condición lógica dada para habilitar el circuito.

Un decodificador con entrada de habilitación puede funcionar como Demultiplexor. Un **Demultiplexor** es un circuito que recibe información de una sola línea y la dirige a una de 2^n posibles líneas de salida. La selección de una salida específica se controla con la combinación de bits de n líneas de selección. El decodificador de la figura 4-5 funciona como demultiplexor de 1 a 4 líneas si E se toma como una línea de entrada de datos, y A y B se toman como entradas de selección.

La variable única de entrada E tiene un camino a las cuatro salidas, pero la información de entrada se dirige a sólo una de las líneas de salida, especificada por la combinación binaria de las dos líneas de selección A y B. Esto se verifica examinando la tabla de verdad del circuito.

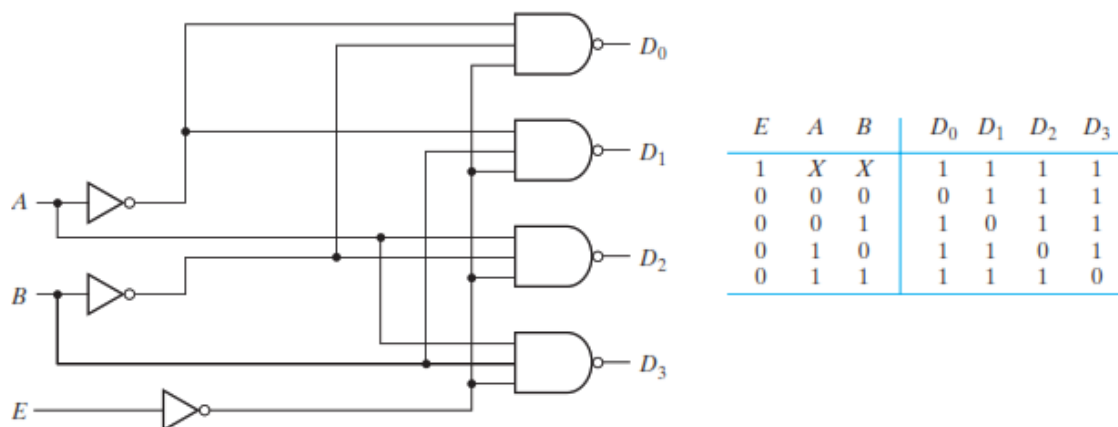


Figura 4-5 Decodificador de 2 a 4 líneas con entrada habilitadora

Por ejemplo, si las líneas de selección $AB=10$, la salida D_2 tendrá el mismo valor que la entrada E , mientras que todas las demás salidas se mantendrán en 1. Dado que se obtienen operaciones de decodificador y demultiplexor con el mismo circuito, decimos que un decodificador con entrada de habilitación es un decodificador/demultiplexor.

Implementar funciones lógicas con decodificadores.

Un decodificador produce los 2^n minterminos de n variables de entrada. Puesto que cualquier función booleana es susceptible de expresarse como suma de minterminos, es posible utilizar un decodificador para generar los minterminos y una compuerta OR externa para formar la suma lógica. Así, cualquier circuito combinacional con n entradas y m salidas se puede implementar con un decodificador de n a 2^n líneas y m compuertas OR. El procedimiento para implementar un circuito combinacional con un decodificador y puertas OR requiere expresar la función booleana del circuito como suma de minterminos. Entonces se escoge un decodificador que genere todos los minterminos de las variables de entrada. Las entradas a cada compuerta OR se escogen de entre las salidas del decodificador, de acuerdo con la lista de minterminos de cada función.

Ilustraremos este procedimiento con un ejemplo que implementa un circuito sumador completo. De la tabla de verdad del sumador completo (véase la tabla 3-2 Semana 10), obtenemos las funciones para el circuito combinacional en forma de suma de minterminos:

$$S(x, y, z) = \sum m(1, 2, 4, 7)$$

$$C(x, y, z) = \sum m(3, 5, 6, 7)$$

Puesto que hay tres entradas y un total de ocho minterminos, se necesita un decodificador de 3 a 8 líneas. La implementación se muestra en la figura 4-6. El decodificador genera los ocho minterminos para x, y, z .

La compuerta OR de la salida S forma la suma lógica de los minterminos 1, 2, 4 y 7. La compuerta OR de la salida C forma la suma lógica de los minterminos 3, 5, 6 y 7.

Si se usan compuertas NAND para el decodificador, como en la figura 4-5, las compuertas externas deberán ser NAND en lugar de OR. El motivo es que un circuito de compuertas NAND de dos niveles implementa una función de suma de minterminos y equivale a un circuito AND-OR de dos niveles.

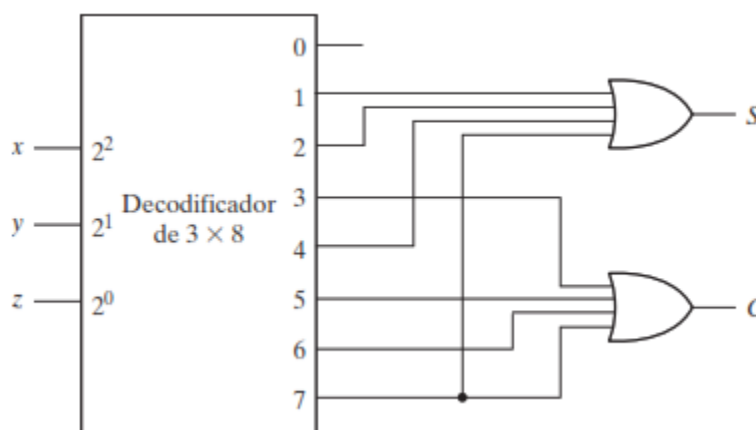


Figura 4-6 Implementación de un sumador completo con un decodificador

Comparadores.

La comparación de dos números es una operación que determina si un número es mayor que, menor que o igual a otro número. Un comparador de magnitudes es un circuito combinacional que compara dos números, A y B, y determina sus magnitudes relativas. El resultado de la comparación se especifica con tres variables binarias que indican si $A > B$, $A = B$ o $A < B$.

El circuito para comparar dos números de n bits tiene 2^n entradas en la tabla de verdad y resulta difícil de manejar incluso con $n=3$. Las funciones digitales que poseen una regularidad inherente bien definida por lo regular se diseñan empleando un procedimiento algorítmico. Un algoritmo es un procedimiento que especifica un conjunto finito de pasos que, si se siguen, producen la solución de un problema. Ilustraremos este método deduciendo un algoritmo para el diseño de un comparador de magnitudes de cuatro bits.

El algoritmo es una aplicación directa del procedimiento que una persona sigue para comparar las magnitudes relativas de dos números. Consideremos dos números, A y B, de cuatro dígitos cada uno. Escribiremos los coeficientes de los números del más al menos significativo:

$$A = A_3A_2A_1A_0$$

$$B = B_3B_2B_1B_0$$

Cada letra con subíndice representa uno de los dígitos del número. Los dos números son iguales si todos los pares de dígitos significativos son iguales; $A_3 = B_3$ y $A_2 = B_2$ y $A_1 = B_1$ y $A_0 = B_0$. Si los números son binarios, los dígitos son 1 o 0, y la relación de igualdad de cada par de bits se expresa lógicamente con una función OR exclusivo así:

$$X_i = A_i B_i + A'_i B'_i \text{ para } i = 0, 1, 2, 3$$

Donde $X_i = 1$ únicamente si los dos bits de la posición i son iguales (es decir, si ambos son 1 o ambos son 0).

La igualdad de los dos números, A y B , se indica en un circuito combinacional con una variable binaria de salida que designaremos con el símbolo $(A=B)$. Esta variable binaria es 1 si los números de entrada, A y B , son iguales, y es 0 en caso contrario. Para que exista la condición de igualdad, las X_i variables deberán ser todas 1. Esto implica una operación AND de todas las variables:

$$(A = B) = X_3 X_2 X_1 X_0$$

La variable binaria $(A=B)$ es 1 únicamente si todos los pares de dígitos de los dos números son iguales.

Para determinar si A es mayor o menor que B , se inspeccionan las magnitudes relativas de pares de dígitos significativos, comenzando por la posición más significativa. Si los dos dígitos son iguales, se comparará el siguiente par de dígitos menos significativos. Esta comparación continuará hasta encontrar un par de dígitos distintos. Si el dígito correspondiente de A es 1 y el de B es 0, concluimos que $A > B$. Si el dígito correspondiente de A es 0 y el de B es 1, sabemos que $A < B$. La comparación sucesiva se expresa lógicamente con las dos funciones booleanas:

$$(A > B) = A_3 B'_3 + X_3 A_2 B'_2 + X_3 X_2 A_1 B'_1 + X_3 X_2 X_1 A_0 B'_0$$

$$(A < B) = A'_3 B_3 + X_3 A'_2 B_2 + X_3 X_2 A'_1 B_1 + X_3 X_2 X_1 A'_0 B_0$$

Los símbolos $(A > B)$ y $(A < B)$ son variables de salida binarias que valen 1 cuando $A > B$ o $A < B$, respectivamente. La implementación con puertas de las tres variables de salida que acabamos de deducir es más sencilla de lo que parece porque implica cierta repetición. Las salidas de desigualdad utilizan las mismas compuertas que generan la salida de igualdad. El diagrama lógico del comparador de magnitudes de cuatro bits se reproduce en la figura 4-7.

Las cuatro salidas X se generan con circuitos NOR exclusivo y se aplican a una compuerta AND para dar la variable binaria de salida $(A=B)$. Las otras dos salidas utilizan las variables X para generar las funciones booleanas que presentamos antes.

Ésta es una implementación multinivel y sigue un patrón regular. El procedimiento para obtener circuitos comparadores de magnitud para números binarios de más de cuatro bits se deduce fácilmente de este ejemplo.

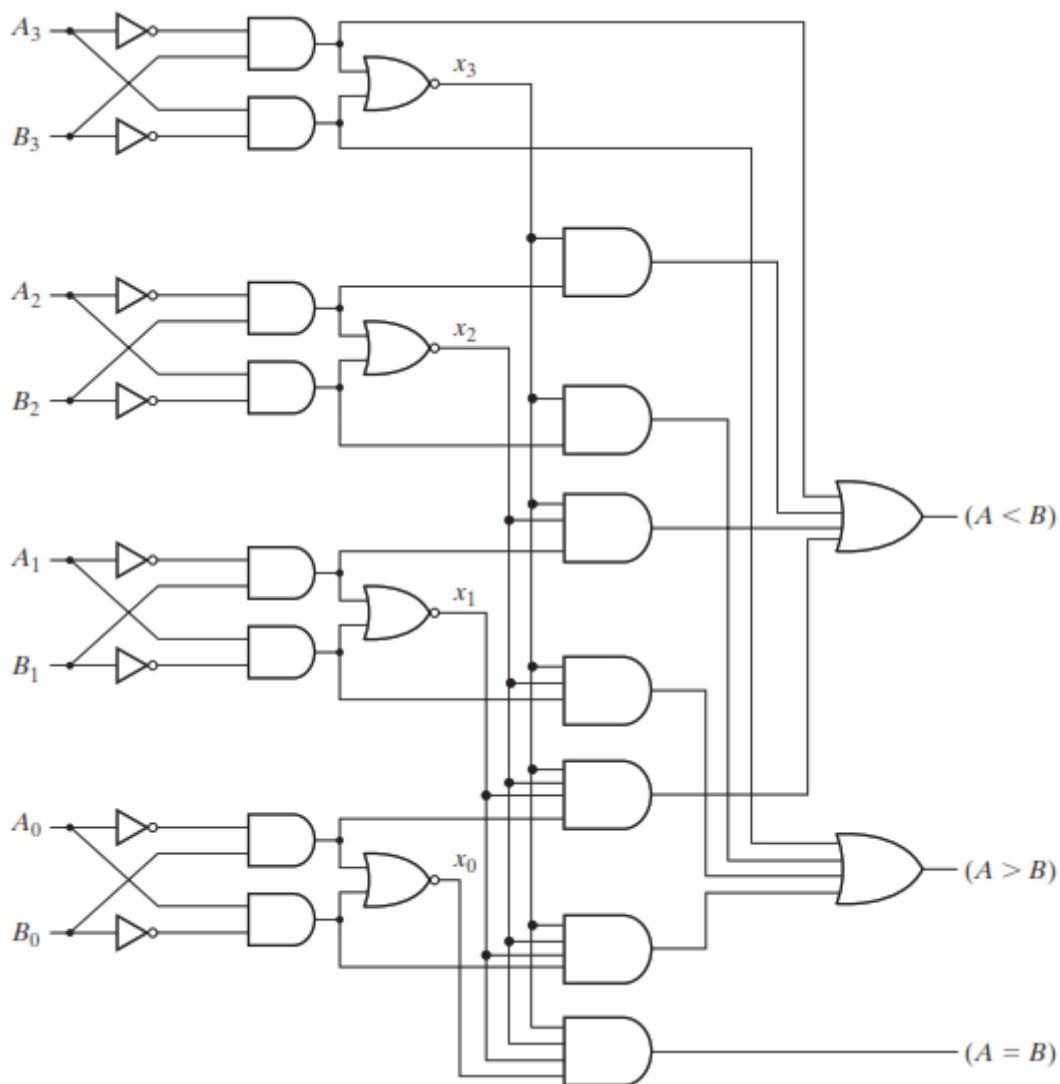


Figura 4-7 Comparador de magnitudes de cuatro bits.